

datalab

取出符号位：

```
x = x >> 31
```

比较一个数与z是否相同

```
x = x & z; //保存所有的1, 如果x中1和z中1的位置一样, 则会生成与z一样的数字  
x = x ^ z; //此时如果x和z一样x就会输出0
```

将ab替换

```
a = (a^b)^a; //a = b;
```

计算绝对值

```
abs = sign ^ (x + sign) //算数右移
```

数字比较时注意数字转换表

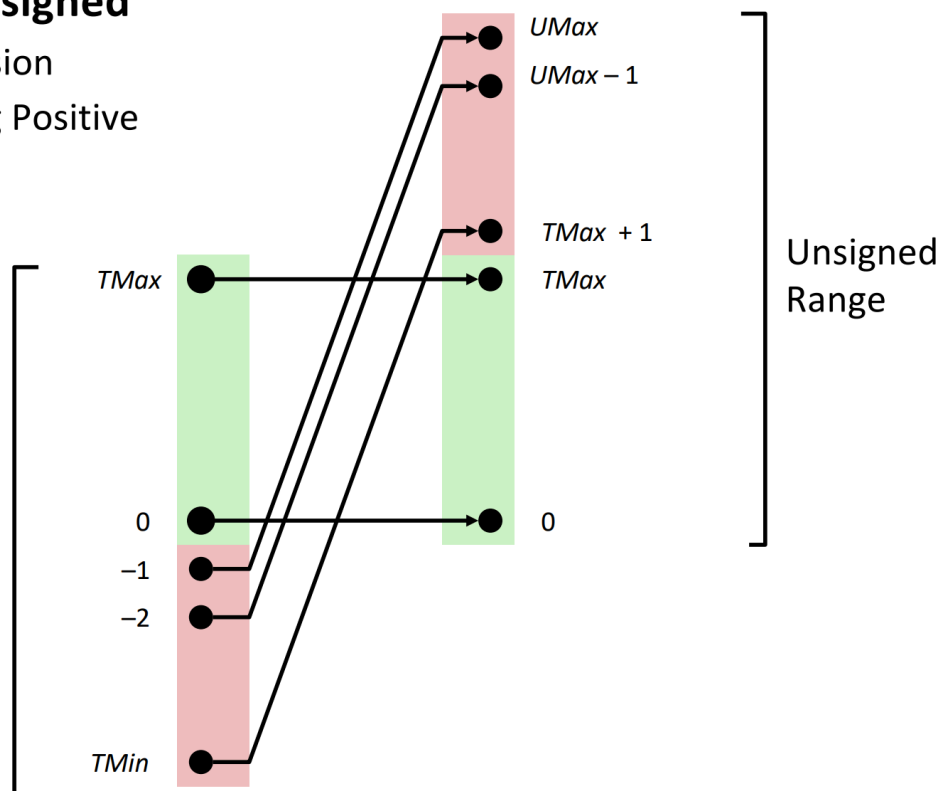
Conversion Visualized

2's Comp. → Unsigned

Ordering Inversion

Negative → Big Positive

2's Complement
Range



bomblab

do.....while循环先运行再测试

while先测试再循环

for循环相当于

```
for(int i = 0; i < N; i++)  
等价于  
while(i < N){  
    .....  
    i += 1;  
}
```

attacklab

注意攻击的时候要按照小端输入

此外，在ret时，rsp已经处于返回位置对应的栈位置，意味着，你的攻击代码应该是buffer加上8的长度（至少）

cachelab

E：一组中有几个项

S：有几组 ($S = 2^s$)

t：对应tag有几位

b：对应block有几位(2^b)

对应一个cache的大小应该是 $S * B * E$

B对应一个cache块有几位

反向无效化应该先把上面的反向无效化

optimlab

没什么好说的，主要方向就是两个：

1. 把所有计算核都用上
2. 减少数据依赖

linkerlab

首先需要注意以下条件：

1. 强符号只能有一个
2. 当有一个强符号和多个弱符号时，弱符号应该被绑定到强符号上

3. 只有多个弱符号时，随机选一个弱符号

还有重定义两种方法

1. PC相对寻址： $addr(symbol) + addend - (addr(s) + offset)$ 其中第一项对应符号位置，addend是为了让它去下一个指令，addr是当时模块的地址，相当于函数前面那堆数字，offset是当前位置相对开始位置的偏移量
2. 绝对寻址： $addr(symbol) + addend$

全局符号global：定义在本模块（看清楚题目说的是哪个文件！！！）内，但可以被其他模块引用，包括非static的函数/全局变量。

b. 外部符号extern：定义在别的模块，在本模块中被引用，用extern 声明。

c. 局部符号local：在本模块内定义的static 全局函数和变量（不一定要全局static）。出现同名static 变量时，编译器分配x.1, x.2 等不同的名称。

d. 函数内部的非静态变量不再符号表中，由stack 管理

malloclab

没有双向链表的时候，首次适配和首次释放的合并都是O(n)的，但是有双向链表的时候合并应该是O(1)的

进程的四个状态

running运行时

block被阻塞

stoppe停止

terminated结束

虚拟内存注意的几点

页表储存页表项，页表项对应物理页号

一般来说页表大小说的就是对应页表项后offset位的情况，那么VPO和PPO的位数就可以依靠页表项确定

页表是虚拟内存向物理内存的映射

一般来说虚拟地址数量等表达表达的是虚拟地址有多少位（4kB对应 2^{13} 也就是13位），页表大小就对应VPO和PPO的大小（128B对应7位）

页表存储了页表项，页表项对应一个页，一个页对应了VPO大小的位数，如果要求需要多少页，只需要知道需要的总共的空间（页表项大小乘以页数）再除以一个页的大小就好（因为页表也是一个页）

页目录索引乘上一个页表项对应的空间加上一级页表基址就能得到一级页表存储我们需求空间的页表项位置